

Remembering Objects

 [_ \(https://github.com/learn-co-curriculum/python-p3-remembering-objects\)](https://github.com/learn-co-curriculum/python-p3-remembering-objects)  [_ \(https://github.com/learn-co-curriculum/python-p3-remembering-objects/issues/new\)](https://github.com/learn-co-curriculum/python-p3-remembering-objects/issues/new)

Learning Goals

- Use class attributes and methods to write durable and powerful code.
 - Use class variables to store instances of a class.
 - Accomplish complex programming tasks using knowledge from previous modules.
-

Key Vocab

- **Attribute:** variables that belong to an object.
 - **Constant:** variable whose value cannot be changed.
 - **Instance:** one specific working copy of a class. It is created when a class's `__init__` method is called.
 - **Class:** a bundle of data and functionality. Can be copied and modified to accomplish a wide variety of programming tasks.
 - **Static:** an attribute or method that cannot manipulate the class or instance it belongs to.
 - **Exception:** an error that occurs during the execution of a program. Exceptions can be anticipated and handled without disrupting the execution of the program.
-

Introduction

Let's say we're building a command line game in which players play various rounds until a final tally determines the winner. Or creating an app in which we want to store a list of all of the users who sign up. Or building a program that helps users track and store the passwords for their various accounts.

In all of these situations, and many more we can imagine, our application needs a way to store or remember a collection of class instances. Whether they are instances of a `Game`, `User` or `Password` class, all of these examples would require our program to keep track of instances that are created.

Luckily for us, Python allows us to do so by using class attributes to store new instances as soon as they are created. Let's take a look together.

Using Class Variables to Store Instances of a Class

Imagine we are building an app that manages a user's music. Our app should keep track of all of the songs a user enters and allow our user to browse their existing songs.

Let's take a look at the following class:

```
class Song:

    def __init__(self, name):
        self.name = name
```

With this code, we can create a new song like this:

```
big_energy = Song("Big Energy")
```

Let's go ahead and create another song:

```
out_of_touch = Song("Out of Touch")
```

Uh-oh. Our user wants to browse their songs now and select one to play. Currently, our code in the `Song` class has no way to keep track of the songs we just created and display them back to the user.

Creating the Class Attribute

Let's take a step back and think about the concept of responsibility. Whose job is it to know about *every instance of the `Song` class*? We have two choices right now: an instance of the song class or the `Song` class itself.

It is not the responsibility of an individual song to know about all of the other songs. Keeping track of all of the songs that it creates, however, fits right into the purview of the `Song` class.

So, how can we tell the `Song` class to keep track of every instance that it creates? We use a class attribute.

Let's create a class attribute, `all`, that will store every instance of the `Song` class.

```
class Song:

    all = []

    def __init__(self, name):
        self.name = name
```

Notice that we set our class attribute equal to an empty list. Lists are perfect for storing collections of related data, so we'll use a list to store our `Song` instances.

Now that our class is *set up* to store the instances that it produces, we have to ask: *how* does it store these instances?

Adding Instances to `all`

Before we can answer this question, we should ask another. *When* should the `Song` class become aware of, or store, an instance of itself?

This should happen at the time of instantiation—when a new song gets created, it should be immediately stored by our `Song` class's `all` attribute.

We can implement this by simply adding the new instance that gets created into the list stored in `all` *inside our `__init__` method*.

Let's take a look:

```
class Song:
```

```
all = []

def __init__(self, name):
    self.name = name
    Song.all.append(self)
```

In `__init__` we use the `self` keyword to refer to the new object that has just been created. Remember that when a new object is instantiated, it creates a new instance of the class and then calls `__init__` on that new instance. So, `__init__` is technically an instance method. Inside an instance method we are in what is called **method scope** and `self` will refer to whichever instance the method is being called on.

We push `self` into the list that is stored in `all`. In this way, the `all` class attribute will point to an ever-growing list that contains every instance of the `Song` class that gets created.

Let's refactor our code again to make sure that class attributes are being handled by class methods, rather than instance methods:

```
class Song:

    all = []

    def __init__(self, name):
        self.name = name
        Song.add_song_to_all(self)

    @classmethod
    def add_song_to_all(cls, song):
        cls.all.append(song)
```

Our Code in Action

Let's see what happens when we actually execute the code we've written:

```
ninety_nine_problems = Song("99 Problems")
```

```
thriller = Song("Thriller")
```

Now that we've created some songs, let's ask our `Song` class to show us all of the instances that we just created:

```
Song.all  
# [<__main__.Song object at 0x104949f70>, <__main__.Song object at 0x104949fa0>]
```

Uh-oh, this list isn't going to impress anybody. Let's make a class method to `print` the names of the songs in `all`:

```
class Song:  
  
    all = []  
  
    def __init__(self, name):  
        self.name = name  
        Song.add_song_to_all(self)  
  
    @classmethod  
    def add_song_to_all(cls, song):  
        cls.all.append(song)  
  
    @classmethod  
    def show_song_names(cls):  
        print([song.name for song in cls.all])  
  
ninety_nine_problems = Song("99 Problems")  
thriller = Song("Thriller")  
Song.show_song_names()  
# => ['99 Problems', 'Thriller']
```

Conclusion

We did it! We used a class attribute to store a collection of instances of that class. We added new instances to this storage container every time a new instance was created with the help of the classname in our `__init__` method. Lastly, we wrote a class method to access and print out the name of each song instance stored in our class attribute.

Resources

- [Python Documentation](https://docs.python.org/3/) ➞ (https://docs.python.org/3/)
- [Classes - Python](https://docs.python.org/3/) ➞ (https://docs.python.org/3/)
- [Python Class Attributes: An Overly Thorough Guide - Toptal](https://www.toptal.com/python/python-class-attributes-an-overly-thorough-guide) ➞ (https://www.toptal.com/python/python-class-attributes-an-overly-thorough-guide)
- [Python's Instance, Class, and Static Methods Demystified - Real Python](https://realpython.com/instance-class-and-static-methods-demystified/) ➞ (https://realpython.com/instance-class-and-static-methods-demystified/)